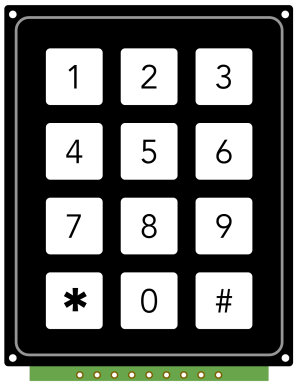
A2 - Keypad



Reference Materials

- STM32L4xxxx Reference Manual (RM) – GPIO
- NUCLEO-L476RG Users Manual (UM) – Pin Diagram (L476RG)
- Keypad Datasheet

Learning objectives:

1. Solidify your understanding of GPIO pins (input/output, pullup/down, read/write).

Keypad

The keypad is a completely passive device made up of 12 (or 16) buttons. To reduce the number of wires and connections, the buttons are connected in a matrix of rows and columns as shown in Figure 1 below. Pressing a button on the keypad does not set or create a high or low voltage. The button press makes a connection between a row and column wire. This means that a button press can not be determined simply by reading from a single GPIO port. Determining a button is pressed requires finding which row and column is connected.

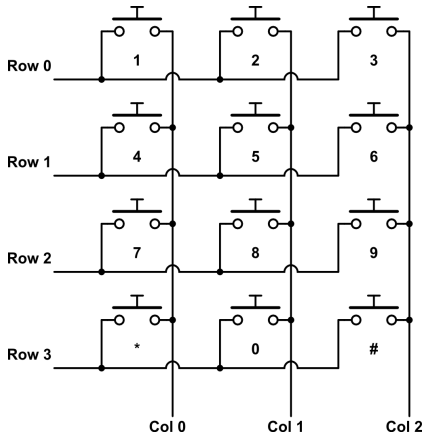


Figure A2.1: 4x3 Keypad Schematic

Detecting Key Presses

Because the STM32L4 does not have dedicated hardware to interface with the keypad, it is limited to interacting with the keypad through polling or interrupts. Polling is performed by repeatedly checking for a button press until one is detected. Determining the button press on a keypad can be performed in 2 stages. The first stage is detecting when a key is pressed while the second is determining which key is pressed. First detecting a key press can be done by setting all of the columns (or rows) high and monitoring the rows (or columns). By setting all of one dimension of the keypad matrix high (columns or rows), any key press will send one of the wires of the other dimension (rows or columns) high when the button press makes a connection. For example, if Column0 – Column2 is set high, pressing button 8 will cause Row 2 to go high because it will make a connection between Column 1 and Row 2. Notice detecting a button is being pressed does not mean it is possible to determine which key was pressed. Pressing button 7 or 9 will also cause Row 2 to go high the same way as button 8. This requires the second stage of the process to determine specifically which button was pressed. (*Hint: This could be utilized to make the keypad interrupt driven*)

In the above example, the difference between buttons 7, 8, and 9 can be made only by determining which column is being connected to Row 2 by the button press. This determination can be made by selectively cycling which columns are set high and reading the rows. If button 8 is pressed, setting Column 0 high while setting the others low will not result in Row 2 being high. This eliminates button 7 as a possibility. Only by setting Column 1 high will Row 2 go high signifying a connection at button 8. Once a button press has been detected in stage 1, the columns (or rows) can be individually cycled while reading the other.

Pull-up / Pull-down Resistors

When no key is pressed on the keypad, the connections between the rows and columns are open. When this occurs, the input signals being read from the rows (or columns) will not be connected to a low or high voltage. This means the input signal will float. Floating signals can create havoc on digital circuits because when the inputs on a digital signal float they may be read as low (0) or high (1). This will create unpredictable behavior in reading keypresses when no keys are pressed. To keep the inputs from floating, the input signals can be forced to go high or low rather than float. This is accomplished by adding resistors that pull the signal up or pull the signal down. These resistors are called pull-up or pull-down resistors. When using pull-up resistors, the button connection would need to drive the input low. When pull-down resistors are used, the button connection would need to drive the input high. In the example behavior described above, the button connections would drive the input high, so pull-down resistors will be needed. Built-in resistors are available on all of the GPIO pins of the STM32L4 and can be configured as either pull-up or pull-down.

Instructions

1. Decide which port(s) to use on the STM32L4 to connect to the keypad. Be mindful that having the rows (4 pins) and columns (3 pins) each being continuous bits on the port will make the program logic simpler to code. The NUCLEO-L476RG has a total of 49 GPIO pins available on

the ST morpho extension pin headers CN7 and CN10. All 16 bits of PA, PB, and PC are accessible. (*Protip: define specific values for items like* ROW_PINS, COLS_PINS, NUM_OF_ROWS, NUM_OF_COLS, ROW_PORT, COL_PORT, and KEYPAD_PORT)

2. Write a keypad function that can determine keypresses and return the value of the key pressed. If no key is pressed, it should return an obvious error value like -1. (*Be mindful of using -1 and unsigned variables.)*

3. Write a main program that uses the keypad function in an infinite while loop and outputs the result when a key is pressed to the LEDs used in the counter program from A1. If no key is pressed the LEDs should not change from what they were previously. Extra keys like * and # will depend on what values you assign to them.

Deliverables

1. Take a video of your working assignment. Share a link to the video in your submission.

2. Submit all of your properly formatted C source code files.

**Properly formatted implies properly tabbed with sufficient comments and adjusted so code doesn't wrap, excess white space removed for a compact presentation, following accepted code formatting standards, header comments at the top of each new file to assist the reader in quickly interpreting what they are looking at, etc.*